

Arquitectura propuesta: AI Use Case Factory con Company Context Engine

React + Vite | Python/FastAPI | Microsoft Foundry Agents | Azure Container Apps | Multilenguaje

1. Proposito del documento

Este documento actualiza la arquitectura de referencia para una plataforma agentica que captura ideas de usuarios de negocio, valida su viabilidad, habilita una revision tecnica posterior, genera un paquete de arquitectura y permite dar seguimiento al valor generado cuando las ideas llegan a piloto o produccion.

El cambio principal es incorporar un Company Context Engine como componente central para que cada cliente pueda desplegar la solucion en su propio ambiente de Azure, cargar o descubrir su contexto empresarial y personalizar la experiencia sin modificar codigo. La solucion tambien debe soportar interacciones multilenguaje para usuarios de negocio, tecnicos y administradores.

2. Vision de producto

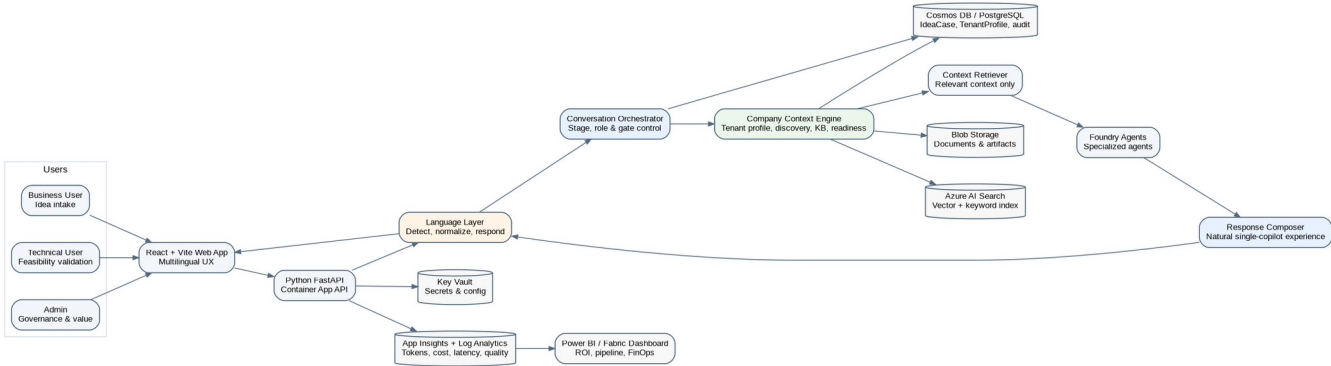
La solucion funciona como una AI Use Case Factory: un acelerador que convierte ideas dispersas en casos de uso trazables, evaluados, priorizados, tecnicamente validados y conectados con valor real de negocio.

La experiencia del usuario debe sentirse como un solo copiloto conversacional, aunque internamente existan agentes especializados, gates formales, contexto del cliente, evaluadores, telemetria, gobierno de prompts y medicion de valor.

3. Principios de diseno

1. Un solo copiloto conversacional para el usuario; multiples agentes especializados por debajo.
2. Company Context Engine como fuente de adaptacion por cliente/tenant.
3. Contexto minimo inicial y enriquecimiento progresivo, evitando formularios largos obligatorios.
4. Separacion estricta entre usuario de negocio, usuario tecnico y administrador.
5. Estado del caso de uso como fuente de verdad, no solo historial conversacional.
6. Soporte multilenguaje con deteccion de idioma, normalizacion interna y respuesta en el idioma del usuario.
7. Salidas estructuradas con JSON/Pydantic para evaluacion, auditabilidad y reduccion de sesgos.
8. Gates formales entre etapas: idea, viabilidad de negocio, validacion tecnica, arquitectura, piloto, produccion y valor realizado.
9. Prompts versionados y gobernados, no quemados en codigo.
10. Telemetria obligatoria por agente, prompt, modelo, usuario, etapa, tenant y caso de uso.
11. Empaquetamiento cloud-native para despliegue automatizado en Azure Container Apps.

4. Arquitectura logica actualizada



Componentes principales

Componente	Tecnologia sugerida	Responsabilidad
Web App	React + Vite	Interfaz multilinguaje para usuarios de negocio, tecnicos y administradores
API	Python FastAPI	Endpoints, autentificacion, autorizacion, orquestacion y telemetria
Hosting	Azure Container Apps	Despliegue serverless de frontend, API y workers
Agent Runtime	Microsoft Foundry Agent Service	Gestion de agentes, herramientas, instrucciones y conocimiento
Company Context Engine	Python services + agentes + RAG	Construccion y recuperacion del contexto especifico de cada cliente
State Store	Cosmos DB o PostgreSQL	TenantProfile, IdeaCase, decisiones, gates, auditoria y metricas
Semantic Search	Azure AI Search	Recuperacion semantica e hibrida sobre conocimiento del cliente
Observability	Application Insights + Log Analytics	Tokens, costo, latencia, errores, calidad, trazas y evaluaciones
Admin BI	Power BI o Microsoft Fabric	Pipeline, valor, consumo, ROI y gobierno de prompts

5. Company Context Engine

El Company Context Engine es el componente que permite que la herramienta sea reusable para multiples clientes. Convierte informacion empresarial dispersa en contexto operacional util para los agentes.

5.1 Modos de captura de contexto

Modo	Uso	Descripcion
Quick Start	Demo o MVP	Datos minimos: empresa, industria, pais, unidades, stack principal
Guided Setup	Cliente nuevo	Formulario corto + preguntas guiadas por Discovery Agent

Enterprise Bootstrap	Cliente enterprise	Carga de documentos, arquitecturas, politicas, OKRs, roadmaps y presentaciones
Progressive Enrichment	Uso continuo	El contexto se enriquece conforme usuarios interactuan y se validan ideas

5.2 Subcomponentes

Subcomponente	Responsabilidad
Tenant Onboarding Service	Crea tenant, configuracion, idioma por defecto y parametros de despliegue
Discovery Agent	Entrevista al cliente para completar informacion critica faltante
Document Ingestion Worker	Extrae contenido de PDF, Word, PPT, Markdown y otros adjuntos
Context Normalizer	Convierte contenido en entidades estructuradas: objetivos, capacidades, sistemas, riesgos
Knowledge Indexer	Publica documentos y fragmentos al indice de Azure AI Search
Context Retriever	Selecciona contexto relevante por rol, etapa, idioma e idea
Readiness Scorer	Calcula madurez de negocio, datos, IA, seguridad y nube
Context Governance	Versiona, aprueba, retira o corrige contexto usado por los agentes

6. Soporte multilinguaje

El sistema debe permitir que un usuario describa ideas, preguntas tecnicas o decisiones administrativas en diferentes idiomas sin romper el estado del caso ni la trazabilidad.

- Detectar idioma por mensaje y conservar el idioma original.
- Responder en el idioma del usuario, salvo que el tenant o el caso indique otra preferencia.
- Mantener campos estructurados normalizados en un idioma canonico interno configurable.
- Guardar tanto texto original como resumen normalizado.
- Usar embeddings multilinguaje o una estrategia de indice que soporte busquedas cross-language.
- Mantener un glosario por tenant para siglas, sistemas internos y traducciones preferidas.

Funcion	Descripcion
Language Detection	Detecta idioma del mensaje y confianza
Locale Resolver	Define idioma de respuesta considerando usuario, tenant y caso
Canonicalization	Normaliza resumen, requisitos y entidades a idioma canonico
Terminology Control	Aplica glosario del tenant para consistencia
Multilingual Retrieval	Recupera contexto aunque el documento este en otro idioma
Response Localization	Genera respuesta natural en el idioma correcto

7. Flujo end-to-end actualizado



- 12. Tenant onboarding: se crea el tenant y se configura idioma, pais, industria y parametros basicos.
- 13. Context bootstrap: formulario corto, Discovery Agent o carga de documentos.
- 14. Company Context Engine: normaliza documentos, crea TenantProfile, indexa conocimiento y calcula madurez.
- 15. Business idea intake: el usuario de negocio describe una idea en su idioma.
- 16. Business validation: se evaluan valor, impacto, riesgo, supuestos y preguntas abiertas.
- 17. Gate 1: si la idea es viable, se congela la version de negocio y se genera un Technical Brief.
- 18. Technical validation: el usuario tecnico valida sistemas, datos, integraciones, seguridad y factibilidad.
- 19. Gate 2: si es viable, se genera paquete de arquitectura y despliegue.
- 20. Pilot / production tracking: se mide avance, adopcion, valor realizado, tokens y costo.
- 21. Context feedback loop: decisiones y resultados alimentan el contexto del tenant.

8. Separacion de experiencias por rol

Rol	Interaccion	Contexto visible	Puede editar	Resultado esperado
Usuario de negocio	Conversacion natural para explicar idea	Contexto de negocio y preguntas abiertas	Campos de negocio	Business Case y viabilidad
Usuario tecnico	Validacion guiada del brief aprobado	Technical Brief y contexto tecnico relevante	Campos tecnicos	Factibilidad y arquitectura candidata
Administrador	Dashboard y gobierno	Todos los casos, trazas y metricas	Estados, prompts, configuracion	Pipeline, ROI, FinOps, auditabilidad

9. Agentes propuestos

Agente	Responsabilidad	Salida principal
Discovery Agent	Construir y actualizar contexto del cliente	TenantProfile parcial/completo
Business Intake Agent	Capturar idea, aclarar problema, usuarios y valor	Idea summary estructurado
Business Viability Agent	Evaluar valor, impacto, riesgos y supuestos	BusinessValidation JSON
Technical Validation Agent	Validar factibilidad, sistemas, datos y seguridad	TechnicalValidation JSON
Architecture Agent	Crear arquitectura objetivo y componentes	ArchitecturePackage
Deployment Plan Agent	Generar roadmap, backlog y pasos de despliegue	DeploymentPlan
Evaluator / Critic Agent	Revisar sesgos, calidad, contexto y consistencia	EvaluationReport
Response Composer	Convertir salidas estructuradas en conversacion natural	Respuesta final localizada

10. Admin Center: valor, trazabilidad y FinOps

Tab	KPIs sugeridos
Pipeline de ideas	Ideas por etapa, area, industria, tiempo promedio por gate, bloqueadores
Valor generado	Ahorro estimado, ahorro real, horas reducidas, ingresos incrementales, ROI
FinOps de IA	Tokens por tenant, agente, modelo, usuario, etapa y caso de uso
Prompt Governance	Versiones activas, costo promedio, calidad, aprobador, fecha de cambio
Auditoria	Decisiones, responsables, evidencia, cambios de estado, evaluaciones

11. Despliegue empaquetado por cliente

Para permitir que cada cliente despliegue la solución en su propio ambiente Azure, se recomienda usar infraestructura como código y configuración por tenant.

- Azure Container Apps Environment
- Container App para frontend React/Vite
- Container App para API Python/FastAPI
- Container App Job o Worker para ingestión y procesamiento de documentos
- Azure Container Registry
- Azure OpenAI o modelos desplegados desde Foundry
- Foundry Agent Service
- Azure AI Search
- Azure Blob Storage
- Cosmos DB o Azure Database for PostgreSQL
- Azure Key Vault
- Application Insights y Log Analytics
- Managed Identity para acceso seguro entre servicios

12. Recomendaciones para GitHub Copilot

22. Crear estructura base del monorepo: /frontend, /api, /workers, /infra, /docs.
23. Implementar modelos Pydantic: TenantProfile, IdeaCase, AgentRunTelemetry, PromptVersion.
24. Crear endpoints FastAPI para tenant onboarding, idea intake, business submit, technical validation y admin metrics.
25. Implementar Language Layer con detección de idioma, locale resolver y campos original_text + canonical_summary.
26. Implementar Company Context Engine con servicios de ingestión, normalización, indexing y retrieval.
27. Crear cliente para Foundry Agents y wrappers por agente especializado.
28. Implementar Telemetry Middleware para capturar tokens, latencia, costo estimado, modelo y prompt version.

- 29. Crear UI React/Vite con tres experiencias: Business, Technical y Admin.
- 30. Crear IaC para desplegar recursos en Azure Container Apps con Managed Identity y Key Vault.

13. Roadmap recomendado

Fase	Alcance	Resultado
MVP 1	Idea intake, business validation, estado del caso, UI simple	Captura y validacion basica
MVP 2	Technical validation, architecture package, response composer	Flujo completo negocio-tecnico
MVP 3	Company Context Engine con quick setup y documentos	Personalizacion por cliente
MVP 4	Admin center con tokens, costo, ROI y prompts	Gobierno y FinOps
MVP 5	IaC multi-tenant y despliegue empaquetado	Producto reusable por cliente

14. Decision arquitectonica clave

El mayor diferenciador no es simplemente usar agentes. El diferenciador es combinar agentes con un Company Context Engine, un Language Layer, un estado de caso versionado y un Admin Center de valor y FinOps. Esto permite que la solucion escale de una prueba de concepto a una plataforma empresarial empaquetable, gobernable y adaptable a cada cliente.